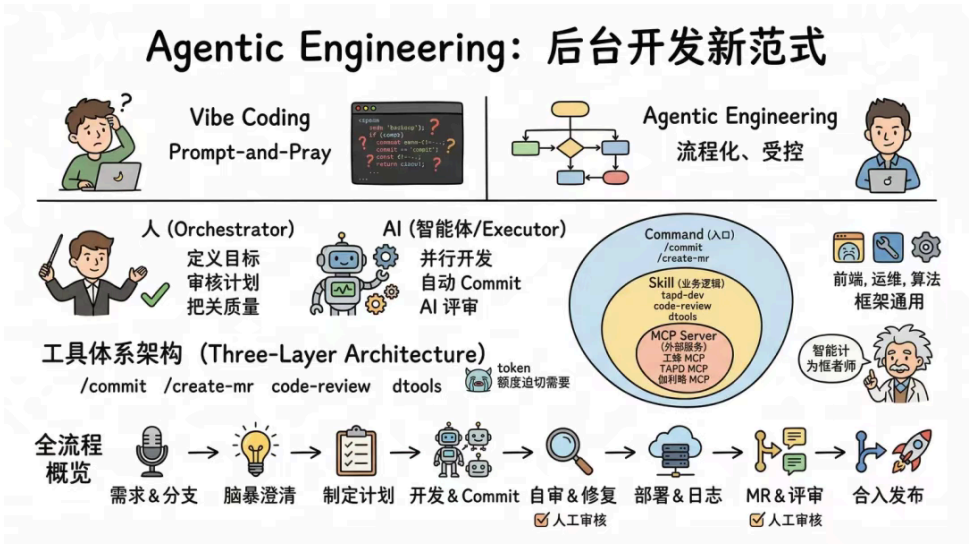
从Vibe Coding到Agentic Engineering：重构后台开发全流程

原创 腾讯程序员 腾讯技术工程 2026年4月17日 02:36 广东 739人



作者：seanguo



引言

做后台开发的同事应该都有这个体会：从接到需求到最终发布，我们要在 PM、GitPlatform、编辑器、DevOps 平台、Galileo 之间来回横跳。每次切换都在丢上下文——刚在 PM 看完需求描述，切到编辑器就忘了某个细节；部署完测试环境去查日志，又得回忆刚才改了哪几行代码。

你可能听过 **Vibe Coding** 这个说法——打开 AI 对话框，用自然语言描述需求，让模型直接生成代码，跑通就算完。原型验证很爽，但一旦要上生产，问题就来了：生成的代码质量不可控、没有审查流程、改完了 commit message 也是乱的。说到底，Vibe Coding 是"提示即祈祷"（prompt-and-pray），你把需求扔给 AI，然后祈祷它别出错。

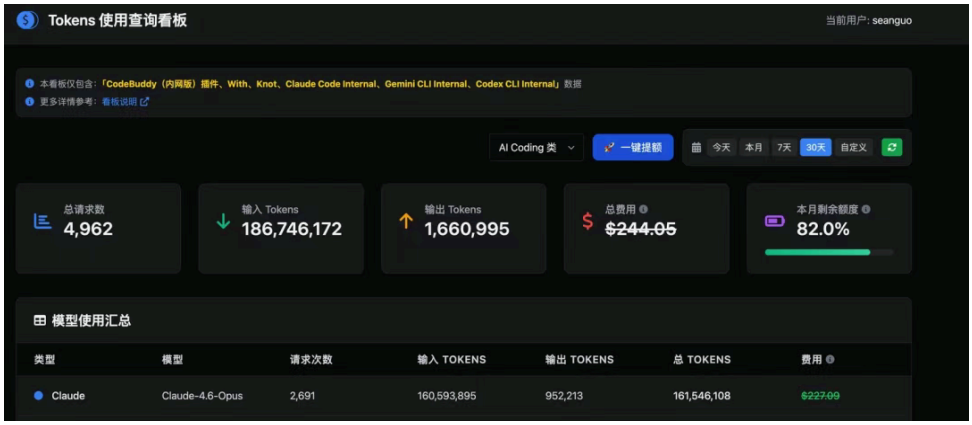
今年行业里逐渐形成了一个更成熟的概念：**Agentic Engineering**（智能体工程）。核心思路是——人负责定义目标、约束条件和质量标准，AI 作为自主智能体在**结构化流程**中执行规划、编码、测试和迭代，每个关键节点都有人工审核。它不是让 AI 随意发挥，而是把 AI 的能力嵌入到一套有纪律的工程体系里。

最近一周，我用 **Claude Code + 自定义 Skill/Command/MCP 体系** 做了一次实践：把从需求到发布的所有环节串到一个终端会话里。AI 全程保持对当前任务的理解，在预设的流程框

架内自主执行；我只需要在关键节点做决策——审批计划、确认部署、审查代码。回过头看，这套东西就是 Agentic Engineering 在后台开发场景的一个落地样本。

这篇文章把整个流程拆开给大家看，从需求到发布每一步怎么跑的，踩过哪些坑，最后沉淀出了什么（~~🔥 token的十大技巧~~）。

~~不过，说实话，虽然过程很完美，但消耗的 token 数量也不容小觑。迫切需要更高的 token 额度子~~



全流程概览

先看效果。下面是整个流程的各阶段概览——从需求到发布，开发者的角色从"亲自执行"变成了"审核确认"：

阶段	核心工具	开发者做什么
① 需求创建 + 分支初始化	`pm-dev`	口述需求
② 需求澄清	`brainstorming`	回答 2-3 个问题
③ 制定实施计划	`writing-plans`	审核计划
④ 并行开发	`executing-plans` + `/commit`	几乎无需干预
⑤ 代码自审	`code-review`	审核报告
⑥ 编译部署	`dtools`	确认部署参数
⑦ 日志排查	`galileo-log-query`	手动触发测试
⑧ 创建 MR	`/create-mr`	确认 MR 信息
⑨ AI 辅助评审	`/review-mr`	审核 AI 评审意见
⑩ 修复评审意见	`/fix-mr`	确认修复方案
⑪ 合入发布	CI/CD	点 Merge + 灰度发布

后面各阶段会逐个展开细节。

工具体系速览

这套工具链分三层，理解了层次关系，后面的内容就好跟了：

层次	说明	示例
Skill（技能）	核心业务逻辑，由系统根据上下文自动触发，或被 Command 调用。每个 Skill 有独立的工具权限白名单和执行流程	`pm-dev`、`git-workflow`、`code-review`、`dtools`、`galileo-log-query`、`git-context`、`wiki-doc`、`service-analyzer`
Command（斜杠命令）	用户通过 `/xxx` 主动调用的入口，轻量级路由，委托给对应的 Skill 执行	`/commit`、`/create-mr`、`/review-mr`、`/fix-mr`、`/analyze-codebase`
MCP Server（外部服务）	通过 Model Context Protocol 连接的外部平台 API，为 Skill 提供数据和操作能力	GitPlatform MCP、PM MCP、Galileo MCP、KnowledgeBase 知识库 MCP、Internal Wiki MCP

此外还有一类来自 `superpowers` 插件的**结构化工作流 Skill**（`brainstorming`、`writing-plans`、`executing-plans`、`subagent-driven-development`、`verification-before-completion` 等），它们定义了从需求澄清到代码交付的标准流程，防止 AI 跳过关键步骤自由发挥。

下面以一个**真实的小变更需求**为例，走完从需求到发布的全流程——「RedeemReward 接口数据上报逻辑变更：无论领取是否成功，都要上报结果，新增 errcode/errmsg 字段」。选这个需求是因为它体量适中（涉及 go mod 依赖更新、结构体扩展、接口逻辑重构），刚好能展示每个阶段的自动化能力，又不会因为业务本身太复杂而分散注意力。



阶段 1：需求获取与分支初始化

> [!info] 使用的工具 **pm-dev** (Skill) — PM 开发初始化工具，两种入口：
>
> 1. 给 PM URL → 自动解析需求 ID，拉取需求标题和正文
> 2. 不给 URL → 通过 brainstorming 澄清需求后，自动调用 PM MCP 创建需求单
>
> 创建完成后自动：建分支（`feature/{username}_{short_id}`）→ 保存需求文档到 `docs/pm/` → 衔接后续开发流程

场景：手头没有现成的 PM 需求单，直接在终端里口述需求就行。

操作：输入 `/pm-dev` 命令并附上需求描述：

```
❯ /pm-dev RedeemReward
接口里的数据上报逻辑变更，无论领取是否成功，都要上报结果
（上报数据新增 errcode，errmsg 字段）。新字段要更新
git.example.com/org/component/report_data/reportstruct go mod 获取。
```

Claude Code 检测到当前在 master 分支，又没有 PM URL，就弹出选择：

```
· 没有检测到 PM 需求单信息，请选择如何继续？ → 自动创建新需求单
```

接下来 AI 会进入 brainstorming 模式澄清需求（详见阶段 2），澄清完自动创建 PM 需求单：

```
● pm - stories_create
  workspace_id: "12345678"
  name: "RedeemReward上报逻辑变更新增错误码"
  description: "RedeemReward 接口的数据上报逻辑变更：无论领取是否成功，都要上报结果..."

✅ 需求单创建成功
  - 标题：RedeemReward上报逻辑变更新增错误码
  - 链接：https://pm.example.com/pm_fe/12345678/story/detail/1012345678001958011
```

口述需求，AI 交互澄清后，自动创建的需求单示例：



随后自动创建开发分支并保存需求文档：

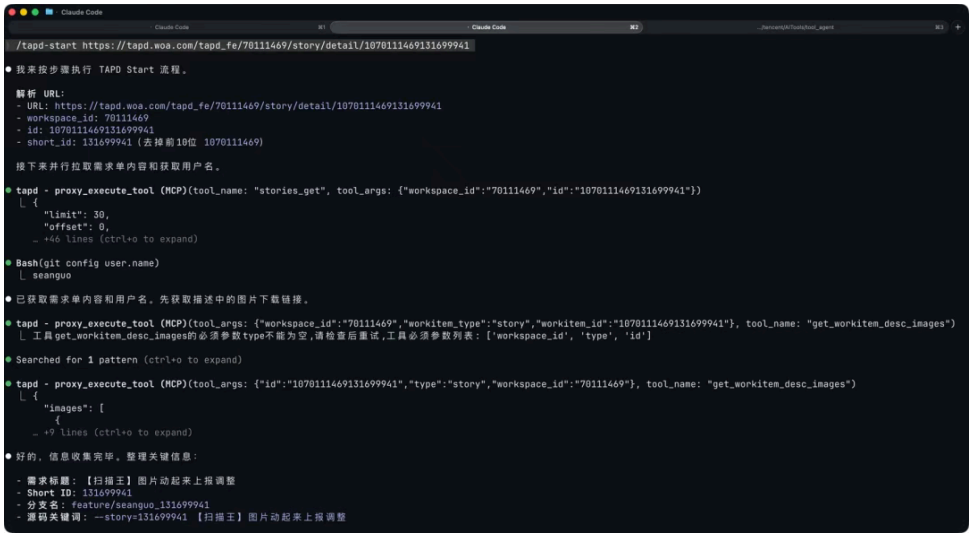
```
● git checkout -b feature/alice_131900001
● Write(docs/pm/131900001.md)  - 保存需求文档
```

效果：一段口述 → PM 需求单 → 规范命名的开发分支 → 需求文档落盘。全程没打开过 PM 页面。

> [!tip] Skill 也可以当命令用 `pm-dev` 是一个 Skill（系统根据上下文自动触发），但你也可以通过 `/pm-dev` 显式调用它。实际上，所有 Skill 都支持以 `/skill-name` 的方式手动触发——当你明确知道要用哪个 Skill 时，直接 `/xxx` 调用比等待自动触发更高效。

> [!tip] 有现成的 PM 需求单？直接提供链接即可：`/pm-dev https://pm.example.com/xxx/story/detail/10xxx`，AI 会自动拉取需求详情并创建开发分支。

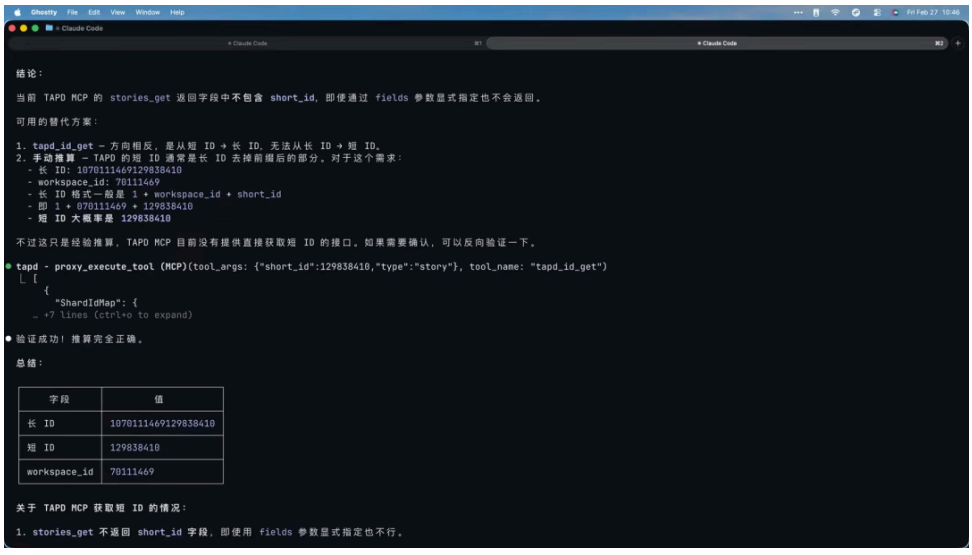
直接拉取需求单示例：



补充一个 AI aha moment 的例子：

我在开发 pm-dev skill 的时候，发现项目管理 MCP Server 并不支持将链接里面的长ID（1012345678001900001）转成 short_id（131900001）。但是想要自动创建特性管理值就需要用到 short_id，那该怎么做呢？

AI 直接推测出了 short_id 和 ID 的关系！（我开发这么久的需求，见过的需求 url & short ID 无数，从来都没有发现两者之间的关系。这个时候我就感觉 AI 是真的能够发现我们肉眼所发现不到的规律。）



阶段 2：交互式需求澄清

> [!info] 使用的工具 **superpowers:brainstorming** (Skill) — 结构化的需求澄清流程。AI 不会直接写代码，而是先探索代码库了解现状，然后通过提问-回答逐步明确需求边界、技术方案和实现路径，最终产出一份双方达成共识的设计方案。**wiki-doc** (Skill) / **knot MCP** — 知识库检索。澄清过程中 AI 可以搜内部 Wiki 文档或通过 KnowledgeBase 知识库做语义检索，为设计决策补充技术背景和历史方案参考。

场景：需求描述就一句话，AI 得先搞清楚代码现状和变更细节才能动手。

> [!info] 链式调用 这个阶段不需要手动触发——`pm-dev` 完成需求创建和分支初始化后，会自动链式调用 `brainstorming` 进入需求澄清。brainstorming 完成后又会接着调用 `writing-plans`（阶段 3），形成 **需求** → **澄清** → **计划** 的自动流水线。

操作：AI 先自动探索项目代码，了解 RedeemReward 接口和上报逻辑的现有实现：

```
● Explore(探索 RedeemReward 和上报逻辑)
  Done (20 tool uses · 93.8k tokens · 56s)
```

然后基于代码理解，抛出关键问题：

```
🤔 新增的 errcode/errmsg 字段是加到现有的 Report_table_001（数据上报表）上，
   还是用一个新的上报表？

❯ 复用 Report_table_001 表，更新 go mod 以后就有新增的 2 个字段了
```

AI 理解后，提出两种实现方案并给出推荐：

```
方案 A（推荐）：在 RedeemReward 末尾统一上报，不提前 return
  将上报逻辑移到函数末尾，把第三方调用的 error 信息传入上报参数。
  - 优点：改动集中，与现有 ReportConsumeCount 风格一致
  - 缺点：需要调整 error 处理流程

方案 B：在每个 return 前都加一次上报调用
  - 优点：不改变原有 return 逻辑
  - 缺点：多处重复调用，容易遗漏
```

```
· 你倾向哪种实现方式？ → 方案 A：统一上报（Recommended）
```

确认方案后，AI 输出完整的设计方案：

```
设计方案：

1. 数据结构变更 – CardReportParam 新增 ErrCode/ErrMsg 字段
2. RedeemReward 流程变更 – 用 defer 确保上报一定执行
3. 变更文件清单：
   ┌──────────────────────────┬──────────────────────────────────────────┐
   |           文件           |                  变更                    |
   ├──────────────────────────┼──────────────────────────────────────────┤
   | go.mod                   | 升级 report_data 版本                    |
   | repo/report/card_report.go | CardReportParam 新增字段，ReportCard |
```

```
|                          | 填充新字段                                  |
| logic/redeem_reward.go   | 重构 RedeemReward，无论成败都上报          |
└──────────────────────────┴────────────────────────────────────────────┘
```

效果：AI 不是闭门造车——先读代码、再提问、再出方案。设计文档自动保存到 `docs/plans/` 并 commit。

> [!tip] 不要重复造轮子 `pm-dev` 早期版本曾经自己实现了头脑风暴/写计划/执行计划的功能，后来发现 `superpowers` 插件已经提供了成熟的 brainstorming、writing-plans、executing-plans 等 Skill，经过大量打磨，效果更好。最终把自制逻辑替换为链式调用现成 Skill。**教训：自定义 Skill 的核心价值是编排和串联，而不是从零实现所有能力。**

> [!tip] 知识库补充上下文 在澄清过程中，AI 可以通过 `wiki-doc` 搜索内部 Wiki 文档，或通过 `knot` 知识库检索已有的技术方案，为设计决策提供依据。

阶段 3：制定实施计划

> [!info] 使用的工具 **`superpowers:writing-plans`** (Skill) — 结构化计划编写。AI 深入读代码细节（结构体字段、错误处理方式、依赖版本），确保计划里每个 Task 都有精确的文件路径、代码变更描述和验证标准。计划自动保存到 `docs/plans/` 并 commit，人工审核通过后才进入执行。

场景：设计方案确认后，AI 自动深入阅读代码，生成可执行的实施计划。

操作：AI 先扎进代码细节——结构体字段、错误处理方式、go mod 版本——确保计划里的代码路径和修改点都是准确的：

```
● Read: repo/report/card_report.go, logic/redeem_reward.go, errs 包
● 查找 Report_table_001 结构体当前字段
● 当前版本 v1.0.180 还没有 errcode/errmsg 字段，更新 go mod 后新版本才会有
```

然后生成实施计划（223 行），自动保存并 commit：

```
● Write(docs/plans/2026-03-03-claimgift-report-errcode.md)

计划拆成 4 个 Task：
  1. 更新 report_data 依赖 — go get 升级版本，验证新字段存在
  2. CardReportParam 新增 ErrCode/ErrMsg — 修改结构体和 ReportCard 函数
  3. RedeemReward 无论成败都上报 — 重构流程，asyncReportRedeemReward 接收 error
  4. 最终验证 — go build/vet
```

> [!warning] 人工审核是关键 这是你审查 AI 方案的第一道关卡。计划里每个 Task 都有精确的文件路径、代码变更描述、甚至具体的代码片段，方便逐条审核。AI 可能会遗漏边界情况或误判依赖关系，这一步不能省。

阶段 4：并行执行开发任务

> [!info] 使用的工具 `superpowers:executing-plans` / `superpowers:subagent-driven-development` (Skill) — 计划执行引擎，两种模式：
>
> - **Sequential**：主 Agent 按顺序逐 Task 执行
> - **Subagent-Driven**（推荐）：为每个 Task 派独立子 Agent 并行执行，完成后自动跑 spec review（功能是否符合计划）和 code quality review（代码质量检查）
>
> `/commit` (Command → `git-workflow` commit 模块) — 智能分析 `git diff --cached`，生成 Conventional Commits 格式的 commit message（`feat` / `fix` / `refactor` 等），交互确认后提交。还会检查是否需要拆分原子提交。

场景：计划审批通过，选择执行方式。

```
· 选择哪种执行方式?  → Subagent-Driven (Recommended)
```

操作：AI 按计划逐个派子 Agent 执行，每个 Task 完成后自动跑 spec review 和 code quality review：

```
● Task 1: 更新 report_data
  ├─ Agent 执行 (5 tool uses · 49s)
  ├─ 新字段: Errcode(string), Errmsg(string), 版本 v1.0.182
  └─ ✅ Spec review 通过

● Task 2: CardReportParam 新增字段
  ├─ Agent 执行 (6 tool uses · 1m 6s)
  └─ ✅ Spec review 通过

● Task 3: RedeemReward 上报重构（核心变更）
  ├─ Agent 执行 (4 tool uses · 6m 25s)
  ├─ ✅ Spec review 通过
  └─ ✅ Code quality review 通过，无 critical/important issues

● Task 4: go build + go vet
  └─ ✅ 全部通过
```

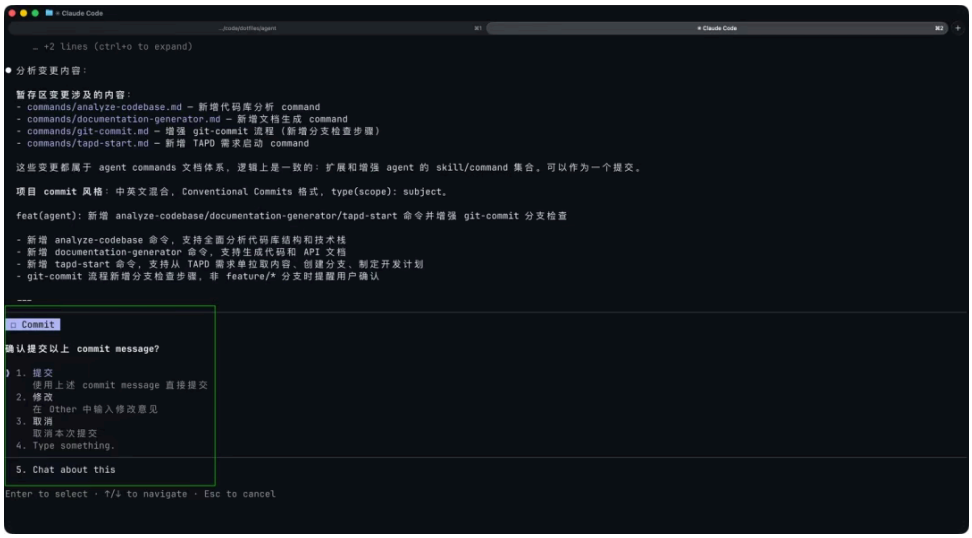
每个 Task 完成后自动生成规范的 Git commit：

```
d81eaee chore: upgrade report_data for new errcode/errmsg fields
18a0975 feat: add ErrCode/ErrMsg to CardReportParam and ReportCard
e978e6b feat: report RedeemReward result regardless of success or failure
```

效果：4 个 Task 全部完成，3 个实现 commit。你只在关键节点确认就行。

```
变更文件汇总：
- go.mod / go.sum — 升级 report_data v1.0.180 → v1.0.182
- repo/report/card_report.go — CardReportParam 新增 ErrCode/ErrMsg，ReportCard 填充
- logic/redeem_reward.go — RedeemReward 无论成败都上报，传入 errcode/errmsg
```

git commit 交互式澄清，自动生成规范的 commit 消息：



企业微信截图_47eb4fc4-7e23-4fa8-b578-bfc7b2a60a5d

规范 git commit 示例，更容易看出变更内容。



subagents 并发任务示例，superpowers:execute-plan 会自动根据需要执行的计划，决定开几个 subagent，全程不用操作。你唯一需要的就是担心 token 额度。



阶段 5：代码审查（自审）

> [!info] 使用的工具 **code-review** (Skill) — Golang 代码审查工具。对 `git diff` 变更做系统化审查，按严重程度（Critical / Major / Minor / Suggestion）和分类标签（业务逻辑 / 安全性 / 性能影响 / 编程规范等 8 类）输出 CR 报告。内置完整的检查清单，覆盖：
>
> - **Critical**：空指针、SQL 注入、数据竞争、资源泄漏、循环中 defer
> - **Major**：错误处理不规范（未用 `errs.Newf/Wrapf`）、嵌套超 4 层、switch 缺 default、并发安全
> - **Minor**：命名规范、import 顺序、魔法数字、函数超 80 行
> - **Suggestion**：可用 `lo` 简化集合操作、可用 `copier` 简化结构体复制、Table-Driven Tests

审查完成后生成 Markdown 报告到 `docs/`，并提供三种后续操作：自动修复所有问题 / 仅保存报告 / 指定修复特定问题（如 C1, M2）。

场景：代码写完了，提交 MR 之前先自己过一遍，在 reviewer 看到之前就消灭明显问题。这一步是"自查"——用 AI 帮你快速扫一遍格式、命名、规范类的低级问题；后面阶段 9 的 AI 辅助评审则是站在 reviewer 的角度做正式审查，两者定位不同。

操作：直接在终端里说就行：

```
❯ 审查一下当前分支相对 master 的代码变更
```

AI 自动加载 `code-review` skill，获取 diff 并执行审查：

```
● git diff master...HEAD — 3 个文件，+21 -11 行

审查报告：
┌──────────┬──────┐
| 级别     | 数量 |
├──────────┼──────┤
| Critical | 0    |
| Major    | 0    |
| Minor    | 1    |
| Suggestion| 1   |
└──────────┴──────┘

m1. [CR-可读性] ChannelID 魔法数字缺少注释
  文件：logic/redeem_reward.go:43
  ChannelID：3 缺少注释说明含义（会员页发放场景）
  建议：添加行内注释或提取为命名常量

S1. [CR-编程规范] Errcode/Errmsg 字段缩进不一致
  文件：repo/report/card_report.go:57-58
  与上方字段的对齐方式不一致
  建议：统一使用 gofmt 标准对齐

● Write(docs/code-review-report-20260303-153022.md) — 保存报告
```

```
· 如何处理审查结果？
  ○ 自动修复所有问题
  ○ 仅保存报告
  ○ 指定要修复的问题（输入编号如 m1, S1）
```

> [!warning] 本文示例中跳过了自审修复 为了在后面**阶段 9（AI 辅助代码评审）**完整展示 `/review-mr` 的工作流——包括 AI 如何发现问题并精确提交行级评论到GitPlatform——这里选了"仅保存报告"，把两个问题留给 MR 评审阶段处理。实际开发中建议自审阶段就修掉所有问题，让 reviewer 聚焦更高层面的设计和逻辑。

效果：`code-review` 相当于一个随时待命的 Golang 审查专家，内置了团队的编码规范和检查清单。提 MR 前先跑一遍，格式、命名、规范类问题基本能清干净，reviewer 就不用在这些细节上浪费时间了。

阶段 6：编译部署到测试环境

> [!info] 使用的工具 **dtools** (Skill) — DevOps 平台 CLI 工具集成。自动从 Makefile 探测 APP/SERVER/ENV/INSTANCE 等参数（找不到时依次查 go.mod、trpcprotocol 协议路径），支持 `dpatch`（包发布）、`bpatch`（二进制发布）、`ipatch`（镜像发布）三种模式。

> 还会自动处理 Mac → Linux 交叉编译（`CGO_ENABLED=0 GOOS=linux GOARCH=amd64`），仓库没有 Makefile 时会自动生成标准化构建/部署模板。

场景：代码开发完毕，部署到 pre 环境跑一下看看。

操作：输入 `/dtools`，AI 自动从 Makefile 中提取部署参数：

```
确认部署参数：
  APP=my_app, SERVER=reward_service, ENV=pre, USER=alice
· 选择哪种发布方式？ → 指定实例发布 (make deploy)
```

AI 执行交叉编译和部署：

```
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -v -o .build/reward_service
dtools bpatch -env pre -app my_app -server reward_service \
  -bin .build/reward_service -user alice \
  -instances "pre.my_app.reward_service.instance001"
```

> [!tip] 自动发现配置过期 部署过程中发现 Makefile 里的实例名 `instance002` 已经过期了，AI 自动跑了 `dtools node ls` 查到当前可用实例是 `instance001`，确认后发布成功，还顺手把 Makefile 里的实例名更新了。这种"顺手修"的能力在日常开发中很实用。

效果：编译 + 部署 + 修正过期配置，一条命令搞定。

dtools发布示例：



阶段 7：日志排查与调试

> [!info] 使用的工具 **galileo-log-query** (Skill) — Galileo日志查询。通过内置 Python 脚本调用 Galileo Log Query API，支持按 target（`BG-PLATFORM.{appid}.{server}`）、

> namespace、时间范围、查询语句、traceID 等条件查日志。自动探测 appid/server（优先从 Makefile、go.mod、trpcprotocol 路径提取），内置防上下文爆炸机制（默认 limit 50，建议先用 `level:error` 缩小范围）。脚本查不到结果时会自动降级到Galileo MCP 的 `ask_question` 接口。

场景：部署后需要验证上报数据是否正确，或者排查异常。这一步目前还是"半自动"的——AI 能帮你查日志、定位问题，但测试请求还得人工手动发，因为 AI 不了解整条调用链的上下游，没法自己构造有效的测试用例。后续计划在代码里加测试入口，让 AI 能直接发 tRPC 请求触发模拟调用。

操作：直接在终端中描述查询需求：

```
❯ 帮我查一下 pre 环境 reward_service 最近 10 分钟的 error 日志
```

AI 通过Galileo API 查询，拿到结果后自动关联代码上下文分析：

```
🔍 问题定位：
  - redeem_reward.go:48 上报时 cfg 为 nil
  - 原因：获取配置失败的路径没有正确传入默认值
  - 建议修复：在 cfg == nil 时使用空 config 而非跳过上报
```

> [!tip] 支持多种查询方式
>
> - 按 trace_id 查询完整调用链
> - 按关键词搜索特定日志
> - 按时间范围 + 日志级别过滤
> - 按 target/namespace 精确定位服务

Galileo日志自助排查示例：



再给一个例子，辅助排查Galileo profile 内容，帮忙快速定位了一个 OOM 问题。



阶段 8：创建 Merge Request

> [!info] 使用的工具 **/create-mr** (Command → `git-workflow` push-mr 模块) — MR 创建全自动化：
>
> 1. 调用 `git-context` 获取 project_id、校验分支命名
> 2. 从分支名提取 PM short_id，通过 PM MCP 拉取需求标题，组装源码关键字（`--story={id} {title}`）
> 3. 分析 `git log origin/master..HEAD` 全部 commit 和 diff，生成 MR 标题和描述（含变更说明 + 影响范围）
> 4. 预览确认后，`git push -u origin` + 调用GitPlatform MCP 创建 MR
>
> **git-context** (Skill) — GitPlatform项目上下文提取，两个复用模块：project-context（解析 project_id、校验分支、检查工作区）和 mr-locate（从 URL 或当前分支定位 MR 并获取元数据）。

场景：测试通过，该提代码了。

操作：输入 `/create-mr`，AI 自动执行完整流程：

```
● git-context:
  - 解析分支名 feature/alice_131900001 → 提取 story ID: 131900001
  - 校验分支命名规范 ✅
  - 通过 PM MCP 获取需求标题

● 分析 git log origin/master..HEAD 的全部 commit

● 预览生成的 MR 信息:
  项目: org/team/project/backend/reward_service
  分支: feature/alice_131900001 → master
```

```
PM: --story=131900001 RedeemReward上报逻辑变更新增错误码

标题：RedeemReward 上报逻辑变更，无论成败都上报并新增错误码
```

确认后自动推送分支并创建 MR：

```
● git push -u origin feature/alice_131900001
● git-platform - create_merge_request

✅ MR 创建成功！
  - 链接：https://git.example.com/.../reward_service/-/merge_requests/64
  - 状态：can_be_merged
  - 必要评审人：bob，charlie（GitPlatform项目已配置默认评审规则，自动分配）
```

自动生成的 MR 描述：

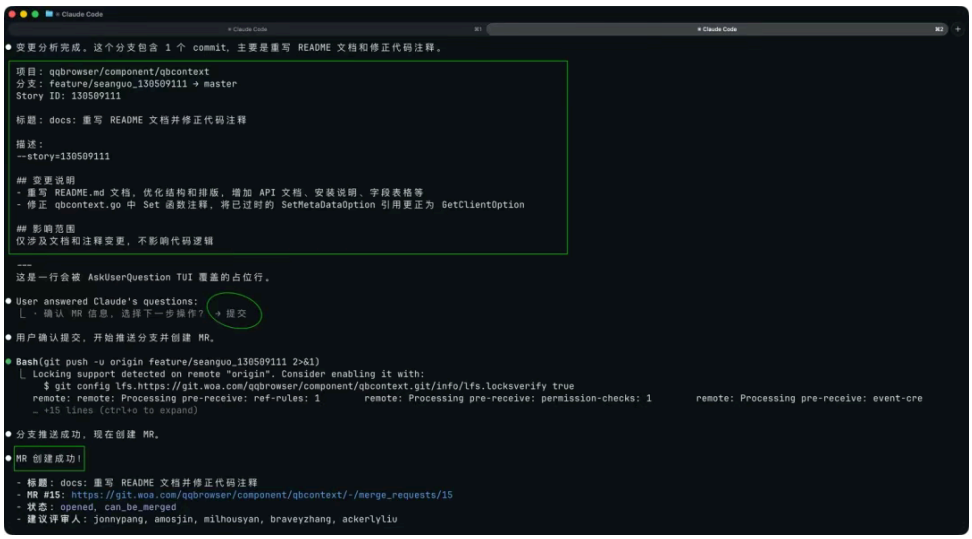
```
--story=131900001 RedeemReward上报逻辑变更新增错误码

## 变更说明
- 升级 report_data v1.0.180 → v1.0.182，获取 Report_table_001 新增的 errcode/errmsg 字段
- CardReportParam 新增 ErrCode/ErrMsg 字段，ReportCard 填充到上报结构体
- RedeemReward 接口无论领取成功或失败都执行数据上报，传入错误码和错误信息
- 获取配置失败时也上报（cfg 为 nil 安全处理）

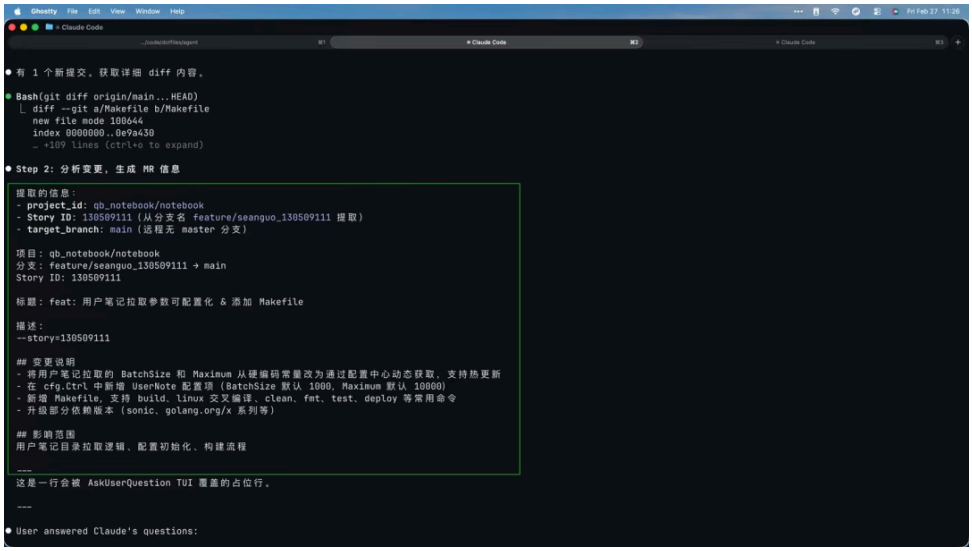
## 影响范围
仅影响 RedeemReward 接口的数据上报逻辑，其他接口的 ReportCard 调用不受影响
（ErrCode/ErrMsg 零值默认 0/""）
```

效果：MR 标题、描述、PM 关联全部自动生成，格式规范、内容完整。

全自动提交 MR 示例，极大缓解了写 MR 描述的痛苦。



企业微信截图_548fd035-e052-4b9a-a260-1615df007faf



GitPlatform MR 页面示例：



阶段 9：AI 辅助代码评审

> [!info] 使用的工具 **/review-mr** (Command → git-workflow review 模块) — AI 代码审查，精确提交行级评论到GitPlatform MR：
>
> 1. 通过 git-context 的 mr-locate 定位 MR（支持从 URL 解析或从当前分支自动查找）
> 2. 加载 code-review 的审查标准（含 Golang 规范、trpc-go 模式、完整检查清单），仅审查新增/修改的行
> 3. 每个问题标注严重程度和分类标签
> 4. 用 git show {source_branch}:{file_path} 验证行号准确性，确保评论定位精确
> 5. 人工审核后批量调用GitPlatform MCP 提交评论（单条失败不阻塞其余）

场景：MR 创建后，reviewer（同事 charlie）用 AI 辅助审查这个 MR。

操作：reviewer 在自己的终端中输入 MR 链接：

```
❯ /review-mr
https://git.example.com/org/team/project/backend/reward_service/-/merge_requests/64
```

AI 自动获取 MR 元数据、拉取 diff、读源文件验证行号，然后输出审查结论：

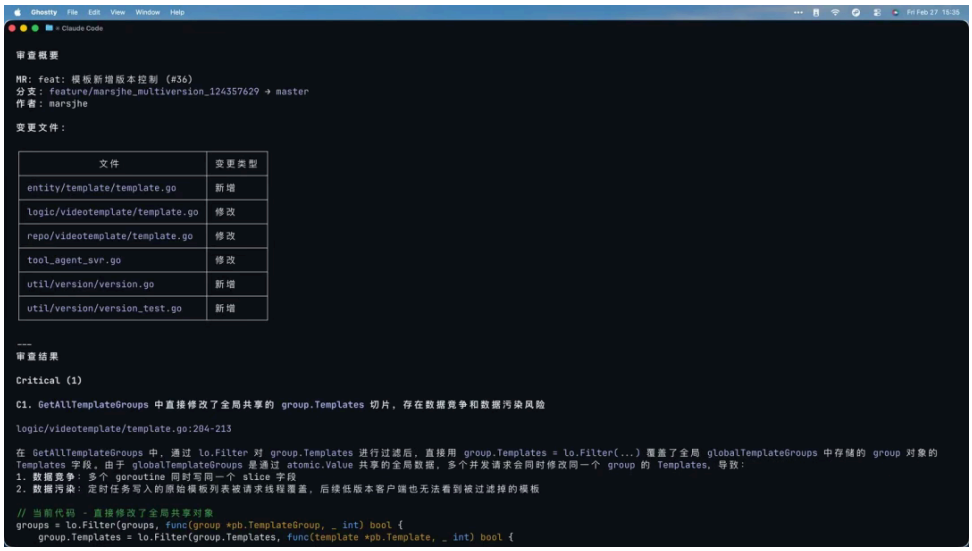
```
变更文件：
  - repo/report/card_report.go (+4 行)
  - logic/redeem_reward.go (+17 -11 行)
  - go.mod (+1 -1), go.sum (+2 -2)

主要逻辑评估：
  - 整体逻辑正确：无论成败都上报，cfg 为 nil 时安全处理 ✓
  - errs.Code(nil) / errs.Msg(nil) 在 trpc-go 框架中 nil 错误返回 0/"" ✓
  - 依赖升级合理 ✓

发现问题：
  Minor 1 个：
  - repo/report/card_report.go:57-58 — Errcode/Errmsg 字段赋值缩进与上方字段不一致

  Suggestion 1 个：
  - logic/redeem_reward.go:43 — ChannelID: 3 魔法数字，原有注释 // 发放场景，3-会员页
    在重构时被误删，建议补回
```

审查概要示例：



审查概要

MR: feat: 模板新增版本控制 (#36)
分支：feature/marsjhe_multiversion_124357629 → master
作者：marsjhe

变更文件：

文件	变更类型
entity/template/template.go	新增
logic/videotemplate/template.go	修改
repo/videotemplate/template.go	修改
tool_agent_svr.go	修改
util/version/version.go	新增
util/version/version_test.go	新增

审查结果

Critical (1)

C1. GetAllTemplateGroups 中直接修改了全局共享的 group.Templates 切片，存在数据竞争和数据污染风险

logic/videotemplate/template.go:204-213

在 GetAllTemplateGroups 中，通过 lo.Filter 对 group.Templates 进行过滤后，直接用 group.Templates = lo.Filter(...) 覆盖了全局 globalTemplateGroups 中存储的 group 对象的 Templates 字段。由于 globalTemplateGroups 是通过 atomic.Value 共享的全局数据，多个并发请求会同时修改同一个 group 的 Templates，导致：
1. 数据竞争：多个 goroutine 同时写同一个 slice 字段
2. 数据污染：定时任务写入的原始模板列表被请求线程覆盖，后续低版本客户端也无法看到被过滤掉的模板

```
// 当前代码 - 直接修改了全局共享对象
groups = lo.Filter(groups, func(group *pb.TemplateGroup, _ int) bool {
    group.Templates = lo.Filter(group.Templates, func(template *pb.Template, _ int) bool {
```

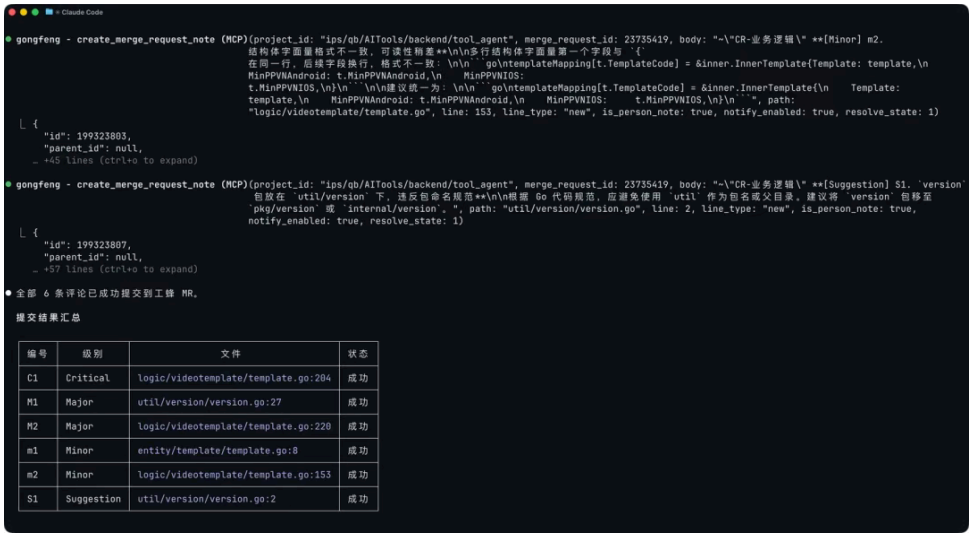
> [!tip] AI 能区分"本次引入"和"历史遗留" 审查过程中 AI 发现 `asyncReportRedeemReward` 函数名带 async 前缀但实际是同步调用，但它主动拉了 master 分支原始代码对比，确认这是历史遗留而非本次引入，所以**没有纳入审查范围**。这个能力很重要——没人喜欢 reviewer 在你的 MR 里提一堆历史债务。

确认后，AI 将 2 条评论精确提交到GitPlatform MR 页面：

```
· 代码审查已完成，请选择评论提交范围：→ Minor + Suggestion（共 2 条）

● git-platform - create_merge_request_note
  ✅ M1 → repo/report/card_report.go:57（含修复代码建议）
  ✅ S1 → logic/redeem_reward.go:43（含补回注释或提取常量两种方案）
```

提交评论示例：极大提高了 CR 效率，不用再一个个去评论了。



● gongfeng - create_merge_request_note (MCP)(project_id: "ips/qb/AITools/backend/tool_agent", merge_request_id: 23735419, body: "~\"CR-业务逻辑\" **[Minor] m2.
结构体字面量格式不一致，可读性稍差**\n\n多行结构体字面量第一个字段与 `{`
在同一行，后续字段换行，格式不一致：\n\n ```go\ntemplateMapping[t.TemplateCode] = &inner.InnerTemplate{Template: template,\n
MinPPVNAndroid: t.MinPPVNAndroid,\n MinPPVNIOS:
t.MinPPVNIOS,\n}\n ```\n\n建议统一为：\n\n ```go\ntemplateMapping[t.TemplateCode] = &inner.InnerTemplate{\n Template:
template,\n MinPPVNAndroid: t.MinPPVNAndroid,\n MinPPVNIOS: t.MinPPVNIOS,\n}\n ```", path:
"logic/videotemplate/template.go", line: 153, line_type: "new", is_person_note: true, notify_enabled: true, resolve_state: 1)

```
└ {
    "id": 199323803,
    "parent_id": null,
  … +45 lines (ctrl+o to expand)
```

● gongfeng - create_merge_request_note (MCP)(project_id: "ips/qb/AITools/backend/tool_agent", merge_request_id: 23735419, body: "~\"CR-业务逻辑\" **[Suggestion] S1. `version`
包放在 `util/version` 下，违反包命名规范**\n\n根据 Go 代码规范，应避免使用 `util` 作为包名或父目录。建议将 `version` 包移至
`pkg/version` 或 `internal/version`。", path: "util/version/version.go", line: 2, line_type: "new", is_person_note: true,
notify_enabled: true, resolve_state: 1)

```
└ {
    "id": 199323807,
    "parent_id": null,
  … +57 lines (ctrl+o to expand)
```

● 全部 6 条评论已成功提交到工蜂 MR。

提交结果汇总

编号	级别	文件	状态
C1	Critical	logic/videotemplate/template.go:204	成功
M1	Major	util/version/version.go:27	成功
M2	Major	logic/videotemplate/template.go:220	成功
m1	Minor	entity/template/template.go:8	成功
m2	Minor	logic/videotemplate/template.go:153	成功
S1	Suggestion	util/version/version.go:2	成功

AI 提交评论的关键，是找准要评论的位置。经过实测，发现 Claude 模型效果最好，DeepSeek 模型经常找错行数，评论到错误的位置。



● User answered Claude's questions:
  └ · 代码审查已完成，共发现 6 个问题（1 Critical, 2 Major, 2 Minor, 1 Suggestion）。请选择要提交到工蜂 MR 的评论范围： → 提交全部评论

● 好的，提交全部 6 条评论到工蜂 MR。先确认关键行号。

● Bash(git show origin/feature/marsjhe_multiversion_124357629:logic/videotemplate/template.go | grep -n "groups = lo.Filter")
  └ 204:   groups = lo.Filter(groups, func(group *pb.TemplateGroup, _ int) bool {

● Bash(git show origin/feature/marsjhe_multiversion_124357629:util/version/version.go | grep -n "return 0")
  └ 27:   return 0 // equal

● Bash(git show origin/feature/marsjhe_multiversion_124357629:logic/videotemplate/template.go | grep -n "func compareVersion")
  └ 220:func compareVersion(qua *quaparser.Qua, innerTemplate *inner.InnerTemplate) bool {

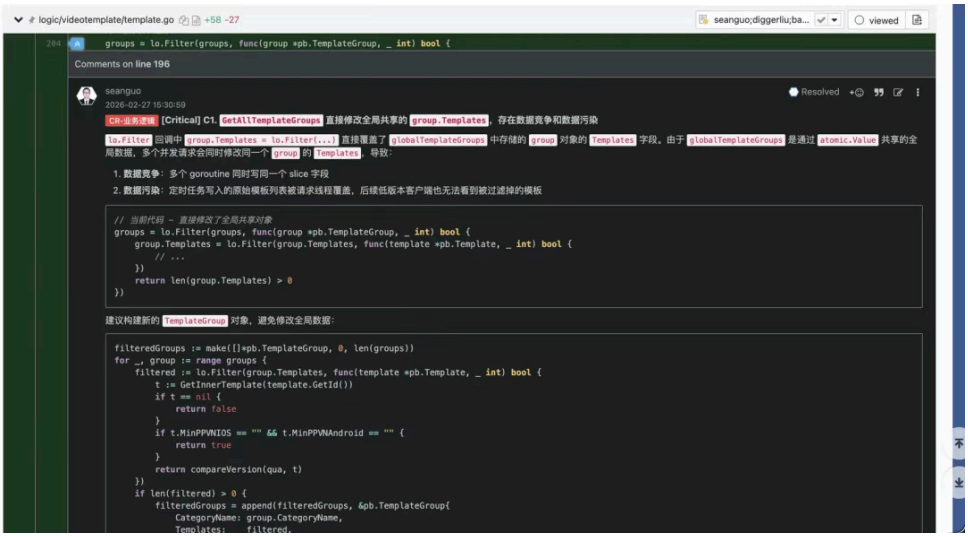
● Bash(git show origin/feature/marsjhe_multiversion_124357629:entity/template/template.go | grep -n "type InnerTemplate")
  └ 8:type InnerTemplate struct {

● Bash(git show origin/feature/marsjhe_multiversion_124357629:logic/videotemplate/template.go | grep -n "templateMapping\[t.TemplateCode\]")
  └ 153:   templateMapping[t.TemplateCode] = &inner.InnerTemplate{Template: template,

● Bash(git show origin/feature/marsjhe_multiversion_124357629:util/version/version.go | grep -n "^package")
  └ 2:package version

● 行号确认完毕，现在并行提交全部 6 条评论。

GitPlatform MR 上面的评论效果：



效果：代码评审从人工逐行看 diff 变成了 AI 初筛 + 人工把关。AI 精确定位到文件和行号，附带代码建议，reviewer 只需审核和决定提交哪些评论。更多真实案例见附录中的「AI 代码评审案例」。

> [!warning] AI 评审有对有错，人工审核不能省 AI 给出的评审意见**不一定都是对的**。它可能误判代码意图、遗漏业务上下文、或者给出看起来合理但实际不适用的建议。上面这个例子里 AI 提的 2 个问题碰巧都是准确的，但在更复杂的 MR 中，我们见过 AI 把正确的错误处理标记为"缺少 error check"、把刻意的空实现标记为"未完成代码"等误判。所以 reviewer 拿到 AI 的审查结果后，**每一条都要过脑子判断**，该采纳的采纳，不靠谱的直接丢掉。

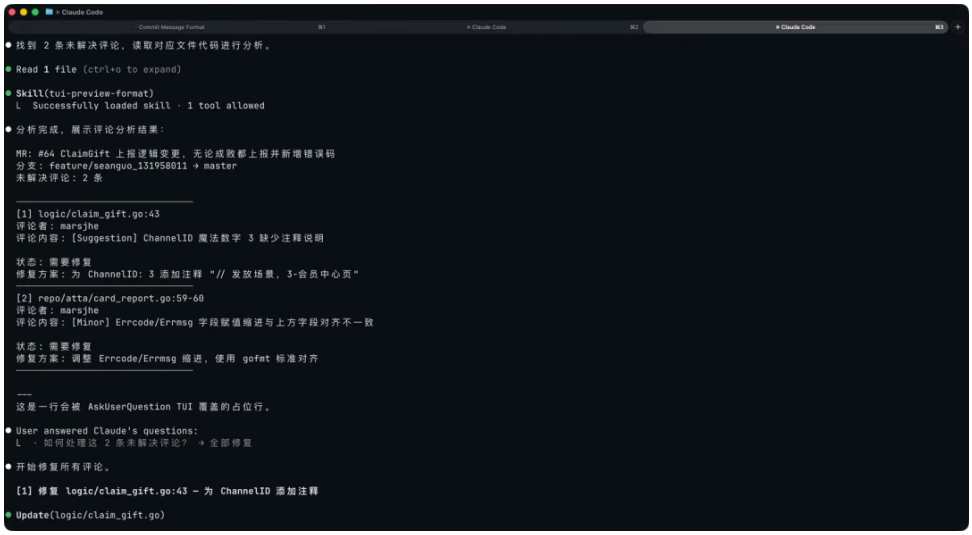
> [!tip] 跨模型审查：避免同一个模型的盲区 一个实用技巧：**谁写的代码，就让别的模型来审查**。同一个模型写的代码再让它自己审，容易陷入相同的思维盲区——它写代码时没考虑到的问题，审查时大概率也发现不了。比如用 Claude 写的代码，可以用 Codex 或 Gemini 来做 AI 评审；反过来也一样。不同模型的知识结构和关注点不同，交叉审查能显著提高问题发现率。

阶段 10：修复 MR 评审意见

> [!info] 使用的工具 **/fix-mr** (Command → git-workflow fix 模块) — MR 评审意见自动修复：
>
> 1. 通过 git-context 定位 MR，自动切到源分支并拉最新代码
> 2. 调用GitPlatform MCP 拉所有未解决评论（ resolve_states=[1] ），提取文件路径、行号、评论内容
> 3. 读对应文件（前后各 10 行上下文），分析问题是否仍存在并生成修复方案
> 4. 交互展示分析结果（含风险评估），确认后逐条修改
> 5. 修改后自动跑 go build ./... 验证编译，并调用GitPlatform MCP 回复每条评论

场景：MR 收到了 reviewer 的评审意见，需要逐条修。

操作：终端里输入 `/fix-mr`，AI 自动定位当前分支的 MR 并拉取未解决评论：




```
● MR: #64 - RedeemReward 上报逻辑变更，无论成败都上报并新增错误码
  未解决评论：2 条

──────────────────────────────────
[1] logic/redeem_reward.go:43
评论者：charlie
评论内容：[Suggestion] ChannelID 魔法数字 3 缺少注释说明

修复方案：为 ChannelID: 3 添加注释 "// 发放场景，3-会员页"
──────────────────────────────────
[2] repo/report/card_report.go:59-60
评论者：charlie
评论内容：[Minor] Errcode/Errmsg 字段赋值缩进与上方字段对齐不一致

修复方案：调整 Errcode/Errmsg 缩进，使用 gofmt 标准对齐
──────────────────────────────────

· 如何处理这 2 条未解决评论？ → 全部修复
```

AI 自动执行修复：

```
[1] 修复 logic/redeem_reward.go:43
  -    ChannelID: 3,
  +    ChannelID: 3, // 发放场景，3-会员页

[2] 修复 repo/report/card_report.go:59-60
  -    Errcode:    cast.ToString(param.ErrCode),
  -    Errmsg:     param.ErrMsg,
  +    Errcode:     cast.ToString(param.ErrCode),
  +    Errmsg:      param.ErrMsg,
```

```
● go build ./...   ✅ 编译通过
● gofmt -d ...     ✅ 格式检查通过
```

修复完成后，通过 `/commit` 提交：

```
fix: address CR comments for RedeemReward report changes

- 为 ChannelID 魔法数字 3 补充注释说明（会员页发放场景）
- 修正 Errcode/Errmsg 字段缩进，与上方字段对齐一致
```

然后自动推送并回复评论。如果后续还有新的评审意见，再跑一次 `/fix-mr` 就行。

效果：评审 → 修复 → 提交 → 回复的完整闭环，全程在终端里完成。

阶段 11：合入发布

> [!info] 使用的工具 CI/CD 流水线 — DevOps 平台自动构建与发布

场景：评审意见全部修完，MR 拿到 Approve。

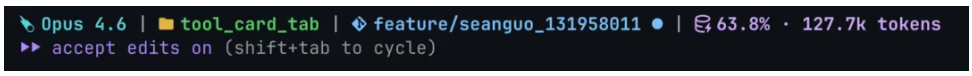
操作：

1. GitPlatform上点 Merge 合入主干
2. 03 流水线自动触发：编译 → 构建镜像 → 推送镜像仓库
3. 按发布流程灰度发布到现网

这一步目前还是纯人工操作——合入和发布涉及灰度策略和线上风险，暂时不打算交给 AI。

效果：从一句口述需求开始，到代码合入主干触发自动发布，整个流程的核心工作都在一个终端会话里完成了。

开发这个需求，消耗的 token 情况如下：



总结

全流程回顾

回到开头那张概览表，这里补充一下各阶段的实际耗时：

阶段	耗时	人工操作
① 需求创建 + 分支初始化	自动	口述需求
② 需求澄清（brainstorming）	~5 min	回答 2-3 个问题

阶段	耗时	人工操作
③ 制定实施计划	~3 min	审核确认
④ 并行开发（4 个 Task）	~10 min	几乎无需干预
⑤ 代码自审	~2 min	审核报告
⑥ 编译部署	~3 min	确认部署参数
⑦ 日志排查	按需	手动触发测试
⑧ 创建 MR	~2 min	确认 MR 信息
⑨ AI 辅助评审	~3 min	审核 AI 评审意见，决定提交哪些
⑩ 修复评审意见（2 条）	~3 min	确认修复方案

整个流程走下来，开发者的工作方式发生了比较大的变化：

传统方式	Claude Code 方式
打开 PM 页面创建需求，手动建分支	口述需求，自动创建需求单和分支
自己拆解任务、逐个实现	AI 制定计划、子 Agent 并行执行、自动 commit
手动 `dtools` 部署，排查实例名过期	AI 自动检测过期配置并修正
手动写 MR 描述，对照 commit 逐条整理	AI 分析全部 commit 自动生成规范描述
在GitPlatform页面逐条看评审、手动改代码	AI 拉取评论、定位代码、生成修复、自动提交

说到底，这就是 **Agentic Engineering** 的核心理念——人是编排者（Orchestrator），AI 是自主执行者：

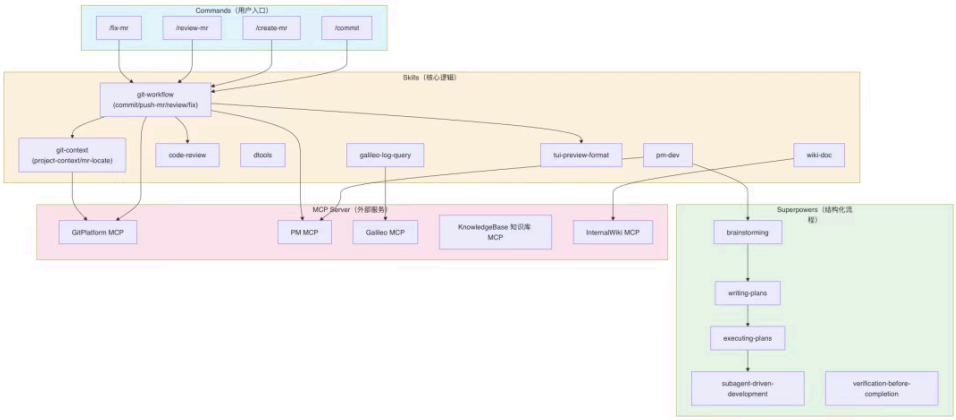
- **人负责**：定义目标、拆解任务、审核方案、把关质量、最终决策——方案选择、计划审核、部署确认、评审把关、合入发布
- **AI 负责**：在结构化流程中自主执行——代码生成、commit 格式化、MR 描述整理、评审意见定位、日志分析……这些重复性高、规则明确的工作

Skill/Command 体系就是那个"结构化流程"——brainstorming 确保先理解再动手，writing-plans 确保先计划再执行，code-review 确保有检查清单而非凭感觉审查。它们把 AI 的能力约束在可控的工程框架里，而不是让 AI 自由发挥然后祈祷结果正确。

这和 Vibe Coding 的本质区别在于：Vibe Coding 依赖运气，Agentic Engineering 依赖流程。每个关键节点都有人工审核，AI 是高效的执行者，不是不受控的自动机。

Skill & Command 的协作架构

理解了全流程之后，来看看这套工具链内部是怎么协作的。整个体系可以用一张依赖图表达：



几个设计上的关键决策：

- **Command 是薄壳**：每个 `/xxx` 命令只有一行——委托给 `git-workflow` 对应模块执行。所以你在对话中说"帮我提交代码"也能触发 commit 流程，不一定非得打 `/commit`。
- **Skill 之间可组合**：`pm-dev` 创建完需求单后自动调用 `brainstorming`，brainstorming 完了接着调 `writing-plans`。`git-workflow` 的每个模块都复用 `git-context` 做前置准备。组合优于重复实现。
- **Superpowers 管纪律**：brainstorming 确保 AI 先理解再动手；writing-plans 确保先计划再执行；executing-plans 确保按步骤推进而不跳跃。这些不是"可选的好习惯"，而是写进 Skill 定义的强制流程。
- **MCP 对用户透明**：你不需要知道"调用GitPlatform API 创建 MR"这件事——Skill 通过 MCP 自动完成，你只看到"MR 已创建，链接是 xxx"。

附录

本文涉及的工具速查

工具	类型	对应阶段	一句话说明
`pm-dev`	Skill	① 需求获取	解析 PM URL 或自动创建需求单，初始化开发分支
`brainstorming`	Superpowers	② 需求澄清	交互式需求探索，先理解再动手
`writing-plans`	Superpowers	③ 制定计划	深入代码生成精确的多 Task 实施计划
`executing-plans`	Superpowers	④ 并行开发	按步骤执行计划，支持子 Agent 并行
`/commit`	Command	④ 并行开发	智能生成 Conventional Commits 格式的 commit
`code-review`	Skill	⑤ 代码自审	Golang 审查，4 级严重度 × 8 类标签
`dtools`	Skill	⑥ 编译部署	DevOps 平台 CLI，自动探测参数并交叉编译

工具	类型	对应阶段	一句话说明
`galileo-log-query`	Skill	⑦ 日志排查	Galileo日志查询，支持多条件过滤
`/create-mr`	Command	⑧ 创建 MR	分析 commit 自动生成 MR 标题/描述
`/review-mr`	Command	⑨ AI 评审	行级代码评审，精确提交评论到GitPlatform
`/fix-mr`	Command	⑩ 修复意见	拉取未解决评论，逐条修复并回复
`git-context`	Skill	⑧⑨⑩	GitPlatform项目上下文提取，被多个工具复用

> 完整的 Skill/Command/Plugin 列表和配置说明，见配置仓库。

AI 代码评审案例

拿一个比本文示例更复杂的 MR（4 个文件，+252 行）来看，AI 审查出了 9 个问题：

级别	数量	典型问题
Critical	1	HTTP 下载无超时控制，goroutine 可能永久阻塞
Major	4	竞态窗口、重复读配置、缺少 HTTP 状态码
Minor	2	错误封装不规范、不必要的导出字段
Suggestion	2	切片预分配、handler 文件拆分

reviewer 审阅后调整了 AI 的部分建议（如将 C1 改为"建议使用 trpc-go http client 代替 net/http"），然后提交了 7 条评论到GitPlatform。

MCP 服务配置

以下 MCP 服务为 Skill 提供外部平台的数据和操作能力，配置一次全局生效：

MCP 服务	用途	依赖的 Skill
GitPlatform MCP	Git 仓库操作、MR 管理	`git-workflow`、`git-context`
PM MCP	需求/缺陷/任务管理	`pm-dev`、`git-workflow`
InternalWiki MCP	Wiki 文档搜索与管理	`wiki-doc`
Galileo MCP	日志查询与分析	`galileo-log-query`
KnowledgeBase MCP	知识库语义+关键词检索	brainstorming 阶段补充技术背景

具体的 MCP 配置命令和 Token 获取方式，见配置仓库。

配置目录结构

```
~/.claude-internal/              # Claude Code 全局配置目录
├── CLAUDE.md                    # 全局指令（操作原则、代码规范、偏好设置）
├── settings.json                # 权限白名单、模型选择、启用插件
├── commands/                    # 自定义斜杠命令（/commit, /create-mr, /review-mr, /fix-mr
└── skills/                      # 技能库（pm-dev, git-workflow, code-review, dtools 等）
```

完整的 Skill/Command/Plugin 列表、官方插件配置、目录详情，均见配置仓库。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容

公众号



阅读 1.8万

留言

写留言

还没有留言，来写首评